Funzioni che restituiscono un risultato e 595 in cascata

Lezione 3 — 2MAK, SST-INF-01c — `return`, la funzione `invia595`, gli array, due e quattro 595 in fila

29/10/2026

Nota

In questa lezione completiamo il discorso sulle **funzioni**: vediamo una funzione che **riceve dei numeri** e **restituisce un risultato** (l'ipotenusa di un triangolo rettangolo). Poi riprendiamo il programma del **74HC595** della dispensa A02 e lo **riorganizziamo**: il ciclo che spedisce gli 8 bit diventa la funzione **`invia595`**, i numeri da mostrare finiscono in un **array**, e infine colleghiamo **due 595 in cascata** (16 LED) e, come sfida, **quattro** (32 LED) usando sempre e solo **tre fili** di Arduino.

Attenzione

Verifica spostata. La verifica di firmware prevista per oggi, 29 ottobre, è stata **annullata** e **spostata a giovedì 5 novembre**. Riguarda **tutto** il programma: saper collegare ad Arduino **LED e pulsanti**, inviare un dato al **monitor seriale** (come l'anno scorso) e in più quello che abbiamo fatto quest'anno: le **funzioni** e la scrittura nel **595**.

1 Ripasso: che cos'è una funzione

Una **funzione** è una **raccolta di comandi**, una specie di **mini programma** che possiamo richiamare dal programma principale. L'esempio che conosciamo bene è il **595**: ogni volta che mettiamo un dato nel 595 dobbiamo dare un **impulso di clock** sul piedino SRCLK. L'impulso si ottiene con **4 righe** di codice: `digitalWrite` alto sul filo del clock, un `delay` di attesa, `digitalWrite` basso e un altro piccolo tempo di sicurezza.

Invece di riscrivere queste 4 righe ogni volta, le abbiamo raccolte nella funzione **`carica`** (il nome lo sceglie chi programma: "carica" sembrava coerente con quello che fa).

```
void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

Nel `loop` basta scrivere **`carica();`**: Arduino esegue questo piccolo programma e, quando ha finito, **torna** a quello che stava facendo.

1.1 Il tipo di ritorno: l'esempio del panino

Una funzione può **accettare dei parametri**, cioè dei dati che le passiamo per **personalizzarne** il lavoro, e può **restituire un risultato**. La forma generale è questa:

```
tipo NomeFunzione(tipo parametro1, tipo parametro2)
{
  ...
}
```

Il nome lo scegliamo noi; **davanti al nome** si scrive il **tipo di dato di ritorno**, cioè il tipo del risultato che la funzione ci restituisce quando ha finito.

Esempio

L'amico che converte i soldi in cibo. Immaginate di "programmare" un amico perché **converta soldi in cibo**: io gli consegno dei soldi, lui (non so come e non mi interessa: magari va in un fast food e compra un panino) mi **ritorna del cibo**. Il bello è che una volta che l'ho "programmato" non ci penso più: gli do i soldi e ottengo il panino. I soldi sono il **parametro**, il panino è il **valore di ritorno**.
E se l'amico prende i soldi e **scappa**, si mangia il panino lui e non mi riporta niente? Anche questo tipo di funzione esiste: è una funzione **`void`**. `void` vuol dire che la funzione **non restituisce niente**: è un tipo "senza dimensione". La nostra `carica()` è proprio così: non riceve niente e non restituisce niente, alza e abbassa un piedino e basta.

I tipi che conosciamo per il risultato sono gli stessi delle variabili: **`int`**, **`char`**, **`float`**, più **`void`** per "nessun risultato". Ci sono funzioni a cui consegniamo dei **numeri interi** e che ci restituiscono un risultato di **un altro tipo**, per esempio un `float`: dipende da quello che devono fare.

2 Una funzione con risultato: l'ipotenusa

2.1 Perché serve: il gioco del Pong

Un esempio che capita spesso. Dobbiamo realizzare il gioco del **Pong** su una **matrice di LED**, cioè una **griglia** di LED che accendiamo in base a quello che deve succedere. Se è acceso un LED e la pallina si sposta, dobbiamo accendere un LED che sta, per esempio, **4 caselle più a destra** e **3 caselle più in su**. Per calcolare la **velocità** con cui "sparare" la pallina ci serve la **distanza** tra i due LED: conosciamo lo spostamento orizzontale e quello verticale, e la distanza si calcola con il

teorema di Pitagora.

Definizione

Teorema di Pitagora. In un triangolo rettangolo con cateti a e b, l'**ipotenusa** c vale

$$c = \sqrt{a^2 + b^2}$$

cioè il quadrato costruito sull'ipotenusa è uguale alla somma dei quadrati costruiti sui cateti.

2.2 Che tipi usare

Creiamo una funzione **`ipotenusa`** a cui **passiamo i due cateti** e che ci **risponde con l'ipotenusa**.

- **Parametri:** se i cateti sono distanze tra **pixel** (o LED), sono **numeri interi**: non esiste mezzo pixel, non posso accendere il pixel 3,5. È come chiedere a qualcuno di sedersi a metà tra due postazioni: o si siede nella prima o nella seconda. Quindi i parametri sono due **`int`**: `int a, int b`.
- **Risultato:** con la **radice quadrata** sarà molto difficile che venga un numero intero, quindi il risultato sarà un **`float`**, un numero con la virgola.

```
float ipotenusa(int a, int b)
{
  int somma = a * a + b * b;
  float ip = sqrt(somma);
  return ip;
}
```

Prima calcoliamo il pezzo sotto radice, `a * a + b * b`: con due interi viene **comunque un intero**, quindi lo mettiamo in una variabile `int somma`. Poi calcoliamo la radice quadrata con la funzione **`sqrt`** (dall'inglese *square root*, "radice quadrata"): anche `sqrt` è una **funzione**, già pronta, che riceve un numero e ne restituisce la radice. Il risultato va in un `float ip`, e con **`return ip;`** lo restituiamo a chi ha chiamato la funzione.

2.3 Usare la funzione nel `loop`

```
void loop()
{
  float x = ipotenusa(3, 4);
  float z = ipotenusa(9, 12);
  float y = ipotenusa(4, 7);
}
```

Che cosa succede quando Arduino arriva a `ipotenusa(3, 4)`?

1. **chiama** la funzione, cioè la **esegue**, e le **consegna** i due numeri: il **3** finisce in `a`, il **4** finisce in `b`;
2. la funzione calcola $3 \cdot 3 = 9$, $4 \cdot 4 = 16$, $9 + 16 = 25$;
3. calcola $\sqrt{25} = 5$ e **riconsegna** il 5;

4. il 5 **va a finire in x**: `x` vale 5, frutto del calcolo fatto dalla funzione.

Con `ipotenusa(9, 12)` il 9 va in `a` e il 12 in `b`: $9 \cdot 9 = 81$, $12 \cdot 12 = 144$, $81 + 144 = 225$, $\sqrt{225} = 15$, quindi `z` vale 15. Con `ipotenusa(4, 7)`: $16 + 49 = 65$ e $\sqrt{65} \approx 8{,}06$, quindi `y` vale circa 8,06: qui la virgola serve davvero. (3, 4, 5 e 9, 12, 15 sono due "terne fortunate" in cui la radice viene intera; di solito non è così.)

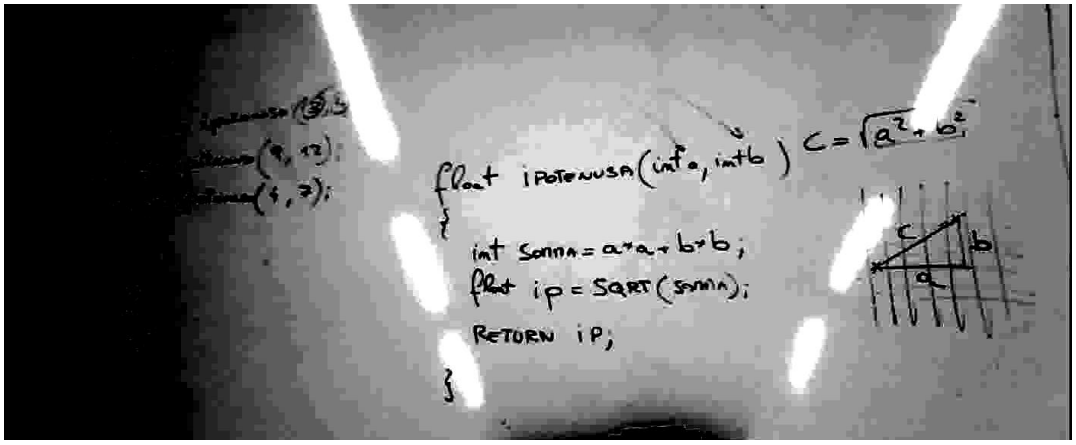


Figura 1: La lavagna: a sinistra il `loop` con le tre chiamate e il conto $81 + 144 = 225$; al centro la funzione `float ipotenusa(int a, int b)`, con `somma`, `sqrt` e `return`; a destra il triangolo con i cateti a, b e l'ipotenusa c.

Attenzione

Il risultato va messo in una variabile del tipo giusto. Se `ipotenusa` restituisce un `float`, non conviene scrivere `int z = ipotenusa(9, 12);`: un **`int` può stare in un `float`**, ma un **`float` non può stare in un `int`** senza perdere qualcosa. Se metto un numero con la virgola in un intero, **perdo tutta la parte decimale**: 8,06 diventerebbe 8.

2.4 Perché conviene

Le variabili `x`, `z`, `y` sono dei **contenitori** in cui mettiamo i risultati di una funzione che **riutilizziamo più volte**: la formula è sempre quella, ma **cambiando i dati in ingresso** cambia il risultato. Senza funzione, per ogni ipotenusa avremmo dovuto riscrivere due istruzioni (somma e radice): sei righe invece di tre. E qui la funzione è piccolina: immaginate un calcolo fatto da **40 righe** di conti complessi, da ripetere ogni volta. Con la funzione si **risparmia** tanto codice scritto e si **abbassa la probabilità di sbagliare**: meno cose scrivo, meno probabilità ho di sbagliare.

Quello che ci interessa capire è questo: a una funzione **posso consegnare dei valori**, e la funzione mi consegna un **risultato che dipende** dai valori che le ho dato.

3 Ripasso: l'AND bit a bit e lo scorrimento

Torniamo al programma del 595 della volta scorsa (dispensa A02). In fondo alla lezione avevamo visto la novità finale: sfruttando l'**AND bit a bit** (`&`) e lo **scorrimento** (`<<`) riuscivamo a "sparare" dentro il 595, **uno alla volta**, i bit che compongono un numero. Avevamo scelto **181** come esempio, ma può essere qualunque numero.

Nota

Le dispense vanno lette e studiate. Prima di rispiegare questo pezzo, in classe abbiamo preso mezz'ora per **aprire e leggere** la dispensa A02, dove tutto questo è già spiegato passo per passo. Una dispensa non si "legge e basta": un capitolo, un pezzo di libro, si **rilegge più volte** finché non si capisce davvero di che cosa sta parlando. "L'ho letta ma non mi ricordo" vuol dire che non è stata capita: è lì che si comincia a studiare.

```
for (int i = 0; i < 8; i++)
{
  digitalWrite(SER, 181 & (B00000001 << i));
  delay(1);
  carica();
}
```

3.1 Il ciclo `for`

`for (int i = 0; i < 8; i++)` **ripete 8 volte** quello che c'è tra le graffe, e a ogni ripetizione la variabile `i` ha un **valore diverso**: la prima volta vale 0, poi 1, 2, 3, al quinto giro vale 4, poi 5, 6 e 7. Quando vale 7 e ha finito, torna in cima, **incrementa** `i` che diventa 8: 8 **non è minore di 8** e il ciclo si chiude. In totale 8 ripetizioni, con `i` che va **da 0 a 7**.

3.2 Che cosa scrive `digitalWrite` a ogni giro

A ogni giro scriviamo sul piedino **`SER`** (il `#define` della volta scorsa, che prima della compilazione diventa il numero **3**) un valore alto o basso, che dipende dal risultato di `181 & (B00000001 << i)`. `B00000001` è il numero 1 scritto in binario; `<< i` lo **sposta a sinistra** di `i` posizioni.

Per capire il risultato dell'AND bisogna scrivere 181 in binario: **181 = 10110101**. Sotto ci scriviamo il numero spostato e facciamo l'AND **colonna per colonna**: $1 \cdot 1 = 1$, mentre se c'è almeno uno 0 il risultato è 0.

`i`	`1 << i` (binario)	`181 & ...`	risultato	sul piedino
0	00000001	181 & 1	1	alto (H)
1	00000010	181 & 2	0	basso (L)
2	00000100	181 & 4	4	alto (H)
3	00001000	181 & 8	0	basso (L)
4	00010000	181 & 16	16	alto (H)
5	00100000	181 & 32	32	alto (H)
6	01000000	181 & 64	0	basso (L)
7	10000000	181 & 128	128	alto (H)

Definizione

Vero e falso per il computer. **Solo lo zero è falso**; **qualunque altro numero è vero**. Quindi 1, 4, 16, 32, 128 sono tutti "veri" e `digitalWrite` mette il piedino **alto** (è come scrivere `HIGH`); 0 è "falso" e il piedino va **basso**.

Dopo ogni `digitalWrite` aspettiamo un attimo e chiamiamo `carica()`, che alza e abbassa SRCLK e fa entrare quel bit nel 595. Così il primo giro carica un alto, il secondo un basso, il terzo un alto, e avanti: in **tre righe di codice** abbiamo buttato fuori tutto il numero 181. Se vogliamo un altro numero, basta **cambiare 181**.

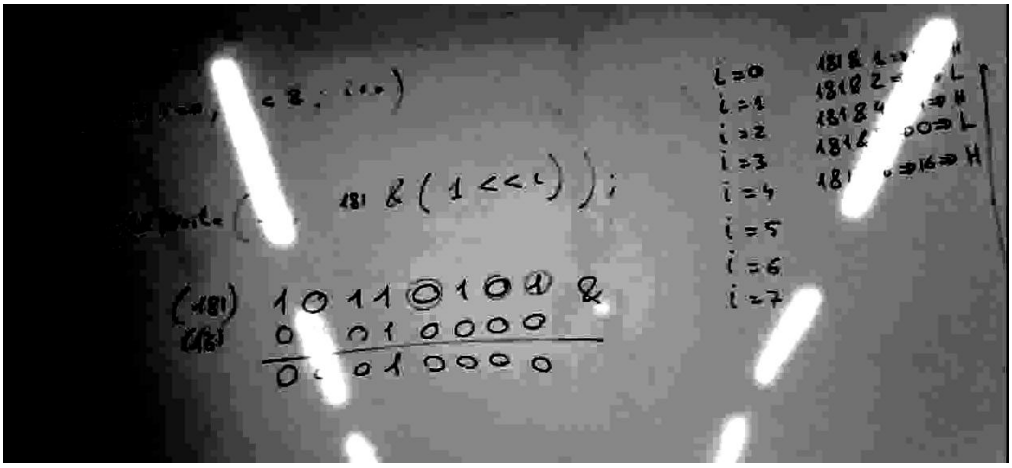


Figura 2: La lavagna: il `for`, la riga `digitalWrite(3, 181 & (1 << i))`, il 181 in binario (10110101) messo in AND con `1 << 4` = 00010000, e a destra i valori di `i` con i risultati: $181\&1 = 1 \Rightarrow$ H, $181\&2 = 0 \Rightarrow$ L, $181\&4 = 4 \Rightarrow$ H, $181\&8 = 0 \Rightarrow$ L, $181\&16 = 16 \Rightarrow$ H.

3.3 Quanto tempo ci mette

Ogni bit richiede `delay(1)` più i due `delay(1)` dentro `carica()`: **3 millisecondi a bit**, quindi $3 \cdot 8 = 24$ ms per tutto il byte, più i 2 ms dell'impulso su RCLK: circa **26 millisecondi** in tutto. Per noi è praticamente **istantaneo**.

3.4 Altri numeri: 15, 192, 255

Se al posto di 181 mettiamo **15**, che in binario è **00001111**, si accendono solo **i 4 LED da un lato**. Per sapere com'è fatto un numero in binario si può usare la **calcolatrice** del computer, oppure fare il conto a mano: si **divide per 2** e si prendono i **resti**, poi si leggono **al contrario**. Con **255** (11111111) si accendono **tutti**: con i resistori da **470 Ω** il 595 regge; con quelli da 220 Ω la corrente totale sarebbe troppa (lo avevamo visto in A02).

Nessuno ci vieta di scrivere il numero **direttamente in binario**, se è più comodo: `B01010101` vuol dire "acceso, spento, acceso, spento…" e si legge subito.

4 La funzione `invia595`

Il passo successivo naturale è prendere questo `for` e **raggrupparlo in una funzione**, così da **togliere dal `loop`** un pezzo di codice che riguarda solo i dettagli del 595. Al posto di 181 vorremmo poter mettere una **variabile**, per esempio `char d = 181;`, che durante il programma cambiamo in base a quello che ci serve. Allora il numero diventa il **parametro** della funzione: i "soldi" che diamo per avere il "panino".

```
void invia595(char d)
{
  for (int i = 0; i < 8; i++)
  {
    digitalWrite(SER, d & (B00000001 << i));
    delay(1);
    carica();
  }
  // attivo RCLK per trasferire i dati dal registro a uscita
  digitalWrite(RCLK, HIGH);
  delay(1);
  digitalWrite(RCLK, LOW);
  delay(1);
}
```

Il `for` è **ricopiato uguale**: dove c'era 181 ora c'è **`d`**. Anche le righe dell'**RCLK** (l'impulso che pubblica il dato sulle uscite) le spostiamo dentro la funzione, perché fanno parte dell'invio. Da questo momento **come è fatto il 595 non ci interessa più**: nel `loop` basta dire **`invia595(numero)`**.

```
void loop()
{
  invia595(181);
  delay(500);
  invia595(42);
  delay(500);
  invia595(255);
  delay(500);
  invia595(B01010101);
  delay(500);
  invia595(B10101010);
  delay(500);
}
```

Il 595 comincia ad **animare la sequenza**: 181, 42, tutti accesi (255), i dispari, i pari. Con 500 ms va un po' veloce: si può portare il `delay` a **1000** per rallentare. E Arduino sta usando **tre fili**, non otto.

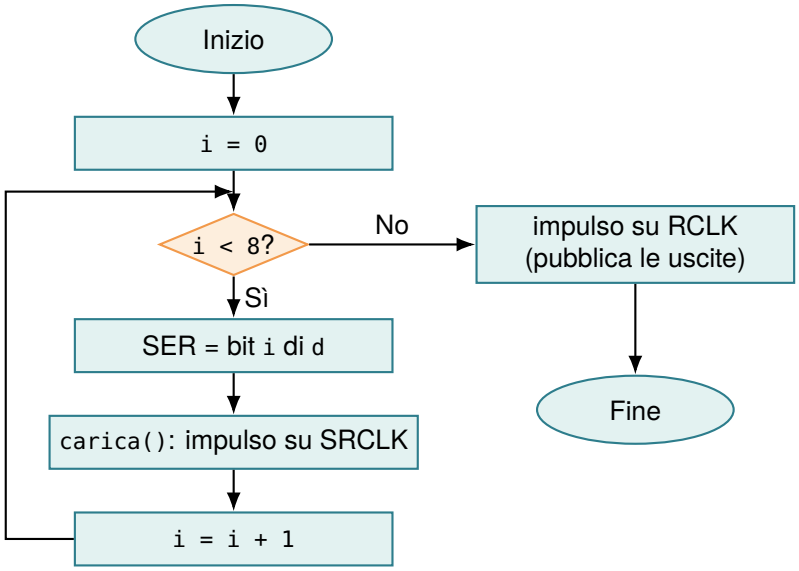


Figura 3: Diagramma di flusso della funzione `invia595(d)`.

Nota

Il tipo `char` e i numeri oltre 127. In questo corso il `char` si usa con valori da 0 a 255. Nel C standard, anche su Arduino, il `char` è con segno ($-128 \ldots +127$) e 0–255 è il range di `unsigned char` (o `byte`): un valore come 181 viene conservato con un altro segno, ma **gli 8 bit sono gli stessi**, quindi l'AND bit a bit e il 595 funzionano comunque. Chi vuole essere preciso può scrivere `byte d` al posto di `char d`.

5 Un array per la sequenza

Vi ricordate gli **array** dell'anno scorso? Un array è una struttura che permette di **raccogliere in un unico oggetto più numeri**. Creiamo un array di **5 elementi** con dentro i numeri della sequenza, e sostituiamo tutte le chiamate del `loop` con un `for`:

```
char sequenza[5] = {181, 42, 255, B01010101, B10101010};

void loop()
{
  for (int i = 0; i < 5; i++)
  {
    invia595(sequenza[i]);
    delay(500);
  }
}
```

Il risultato è **lo stesso**: butta fuori il 181, aspetta, poi il 42, aspetta, poi il 255, e così via (con il `delay` portato a **1000** si vede meglio). Ma adesso, se domani decido di mettere **50 sequenze**, aggiungo 50 numeri nell'array (e cambio il 5 del `for`) **senza scrivere nessuna riga di codice in più**, solo i dati da buttare fuori. In aula l'array è stato davvero allungato a 50 numeri:

```
char sequenza[50] = {181, 42, 255, B01010101, B10101010,
                     B00000000, B11111111, B00001111, B11110000,
                     ... altri numeri, fino a 50 ...};

void loop()
{
  for (int i = 0; i < 50; i++)
  {
    invia595(sequenza[i]);
    delay(1000);
  }
}
```

Il 595 li butta fuori **tutti, uno alla volta**, in sequenza. In questo modo si crea il **gioco di luci** che si preferisce.

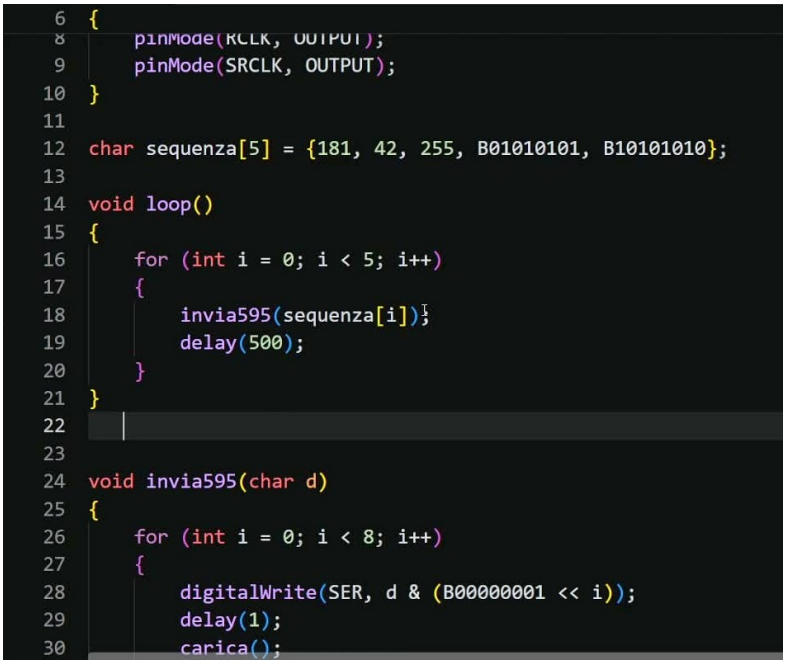


Figura 4: Il codice in Visual Studio Code: l'array `sequenza[5]`, il `loop` con il `for` da 0 a 4 e l'inizio della funzione `invia595(char d)`.

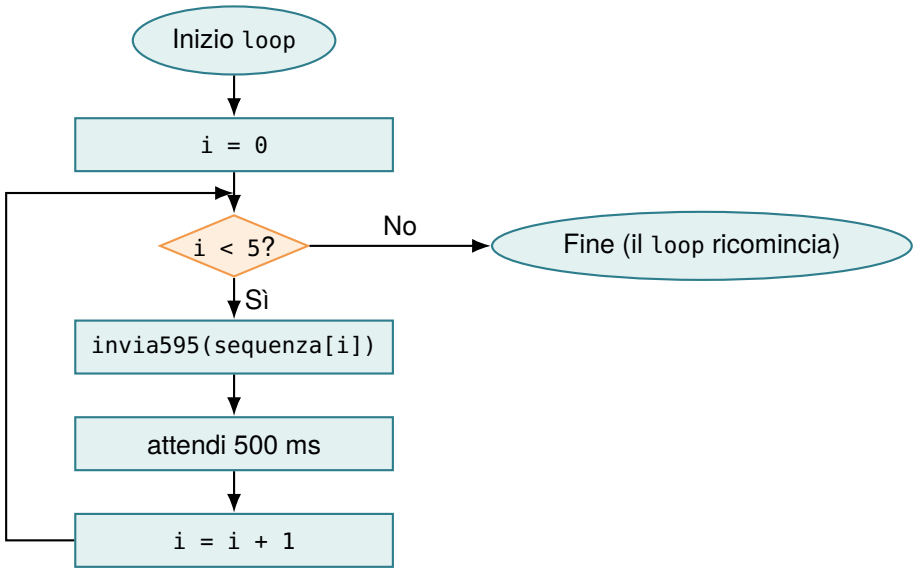


Figura 5: Diagramma di flusso del `loop` con l'array: `i` indica la posizione nell'array, `sequenza[i]` è il numero che sta in quella posizione.

6 Più 595 in cascata

Qui stiamo pilotando un solo 595; il **Pong** che abbiamo nell'altro laboratorio ne ha **molti di più** (circa 128), messi **uno dietro l'altro**: si dice **in cascata**. I collegamenti sono questi:

- i fili **RCLK** (piedino 12) e **SRCLK** (piedino 11) si collegano **in parallelo** a tutti i 595: il 12 di uno va al 12 dell'altro, l'11 all'11;
- il filo del dato invece passa **in serie**: da Arduino **entra nel piedino 14** (SER) del primo 595, **esce dal piedino 9** (Q7', l'uscita seriale) ed entra nel 14 del 595 successivo, e avanti così.

In questo modo se ne possono collegare **quanti se ne vuole**, sempre con **tre fili** di Arduino.

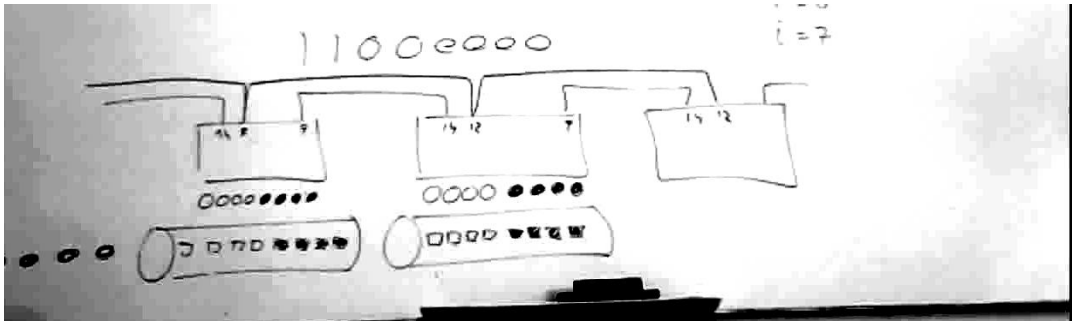


Figura 6: La lavagna: tre 595 in cascata (in alto, i piedini 14, 12, 11 e 9: il 9 di uno va al 14 del successivo, il 12 e l'11 sono in comune) e, sotto, i "tubi" con le palline: quando entrano altre 8 palline, le prime 8 vengono spinte nel tubo successivo.

6.1 Montare il secondo 595 su Tinkercad

Il circuito si espande **ricopiando tutto** il blocco 595 + 8 LED + resistori su una seconda breadboard e facendo i **collegamenti da un circuito all'altro**: piedino 9 del primo al 14 del secondo, 11 con

11, 12 con 12, e naturalmente **l'alimentazione** (5 V e GND) anche alla seconda breadboard.

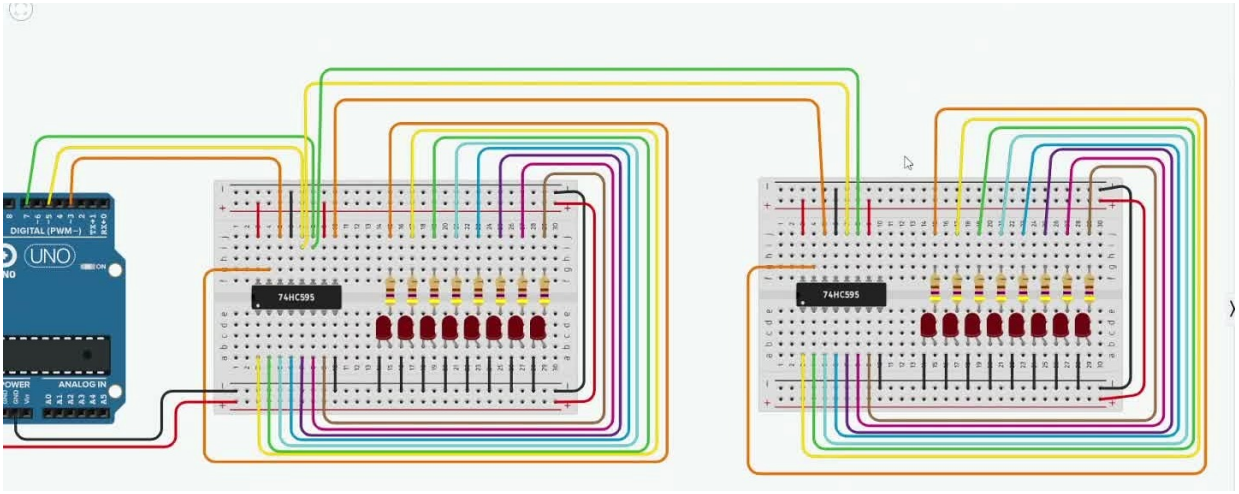


Figura 7: Due 74HC595 in cascata su Tinkercad, ciascuno con 8 LED: Arduino è collegato solo al primo; il secondo riceve il dato dal piedino 9 del primo e i due clock in parallelo.

Attenzione

Ricordarsi l'alimentazione. Nella prima prova il secondo 595 non funzionava: ci si era concentrati sui fili tra i due integrati e **non era stata portata l'alimentazione** alla seconda breadboard. Su Tinkercad l'integrato senza alimentazione non "esplode", semplicemente **non funziona**.

6.2 Che cosa succede con due 595: i due tubi

Immaginiamo i due 595 come **due tubi messi in serie**, ciascuno con 8 finestrelle. Buttiamo dentro 8 palline (4 rosse e 4 bianche): finiti i primi 8 bit, le palline **riempiono il primo tubo**. Se adesso buttiamo dentro **altre 8 palline**, queste **spingono avanti** le prime, che vanno a distribuirsi nel **secondo tubo**, e prendono il loro posto nel primo.

Quindi:

- il **primo** byte inviato finisce, alla fine, nel **secondo** 595 (il più lontano da Arduino);
- l'**ultimo** byte inviato resta nel **primo** 595.

Se mandiamo **sempre lo stesso numero**, per esempio 15, vediamo la stessa sequenza (spento $\times 4$, acceso $\times 4$) su **tutti e due**: il `loop` si ripete e il secondo invio spinge il primo nel secondo 595.

Con due numeri diversi si vede bene: inviamo prima **224** (11100000, tre LED accesi e cinque spenti), poi **15** (00001111). Il 224 entra nel primo 595, poi arriva il 15 che lo **spinge nel secondo**: alla fine il 15 sta nel 595 di **sinistra** (il primo, vicino ad Arduino) e il 224 in quello di **destra**.

6.3 Il "flickering" e dove mettere RCLK

Con l'impulso su RCLK dentro `invia595`, i LED **lampeggiano**: si vedono dei **lampi scuri**. Questo si chiama **flickering** ed è un piccolo **errore di programmazione**: ogni volta che buttiamo dentro

un numero, lo **pubblichiamo** subito sulle uscite. Prima pubblichiamo dopo il 224 (che per un attimo appare nel primo 595), poi di nuovo dopo il 15. La **pubblicazione** va fatta **una volta sola**, **dopo** aver buttato dentro **tutti** i dati.

La soluzione è **togliere l'impulso su RCLK** da `invia595` e metterlo nel `loop`, **dopo l'ultimo invio**: adesso i LED non lampeggiano più e sono **belli stabili**.

6.4 Una sola funzione per tutti e due i clock: `carica(pin)`

Guardando il codice si nota una cosa: la sequenza `digitalWrite` alto, `delay`, `digitalWrite` basso, `delay` è **la stessa** per SRCLK e per RCLK. Cambia **solo il nome del piedino** (il 7 per SRCLK, il 5 per RCLK). In questo caso i piedini sono due, ma immaginate di averne 18 da accendere e spegnere: non si scrive 18 volte la stessa cosa. Allora **standardizziamo**: rendiamo la funzione `carica` **personalizzabile** aggiungendo un **parametro**, il **piedino** su cui dare l'impulso.

```
void carica(char pin)
{
  digitalWrite(pin, HIGH);
  delay(1);
  digitalWrite(pin, LOW);
  delay(1);
}
```

Al posto di `SRCLK` dentro la funzione c'è la parola **`pin`**, e quando la chiamiamo le diciamo quale piedino usare: **`carica(SRCLK)`** a ogni bit, **`carica(RCLK)`** una volta alla fine per pubblicare. Questa modifica **non cambia il funzionamento**: abbiamo solo portato fuori dalla funzione l'informazione del piedino, che prima era scritta dentro. In cambio abbiamo **ridotto il codice di altre tre righe**: adesso è scritto in maniera "seria", nessun programmatore farebbe di meno. Ci siamo arrivati un passo alla volta.

6.5 Il programma completo per due 595

Questo è il programma pubblicato su Classroom (file `doppio595`). Per controllare di avere la versione aggiornata: deve esserci **`carica(RCLK)`** nel `loop`.

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}

void loop()
{
  invia595(224);
  invia595(15);
  // attivo RCLK per trasferire i dati dal registro a uscita
  carica(RCLK);
  delay(500);
}

void invia595(char d)
{
  for (int i = 0; i < 8; i++)
  {
    digitalWrite(SER, d & (B00000001 << i));
    delay(1);
    carica(SRCLK);
  }
}

void carica(char pin)
{
  digitalWrite(pin, HIGH);
  delay(1);
  digitalWrite(pin, LOW);
  delay(1);
}
```

Attenzione

Errori tipici visti ai posti.

- Un numero vecchio rimasto nel codice (per esempio un `181` dimenticato al posto di `d` dentro `invia595`): la funzione deve usare **il parametro `d`**, in cui viene copiato il numero passato (224, 15...).
- `void carica()` **senza** `char pin` tra le parentesi, mentre nel resto del codice si chiama `carica(SRCLK)`: se la funzione non dichiara il parametro, il programma non si compila.
- Fili che nel disegno **passano sopra** una fila della breadboard ma **sono collegati** dove non dovrebbero: un filo appoggiato su un foro è collegato. Se nello schema non c'è, va cancellato.

Il modo più veloce per controllare: **copiare tutto** il codice pubblicato su Classroom (Ctrl+A, Ctrl+C), incollarlo al posto del proprio e simulare. Se così funziona, l'errore era nel codice; altrimenti è nel circuito.

Esempio

LED “quasi spenti”: cercare il guasto con l’oscilloscopio. Se alcuni LED restano leggermente accesi o si comportano in modo strano, su Tinkercad si può usare l’**oscilloscopio**: il filo nero al **negativo**, il rosso sull’**uscita** da controllare. Attenzione alla scala: con **10 µV per divisione** un segnale di 22 µV sembra grande, ma in realtà è **zero** (una piccola corrente di deriva). Se invece si leggono circa **2 V fissi**, l’uscita non è né alta né bassa: un livello che **non accende** bene il LED. In un caso così il guasto era nel circuito: al 595 **mancava il collegamento a GND**. Aggiunto il filo, le uscite tornano a 0 V o 5 V e i LED si comportano come previsto.

7 La sfida: 32 LED con quattro 595

Ultima parte della lezione: pilotare **32 LED**, cioè **quattro 595** in cascata, e far vedere **quattro numeri diversi** su quattro gruppi da otto, in circa 40 minuti. Il circuito si ottiene ripetendo il collegamento in cascata (9 → 14, clock in comune, alimentazione a tutte le breadboard); nel codice basta **inviare quattro numeri** prima di pubblicare.

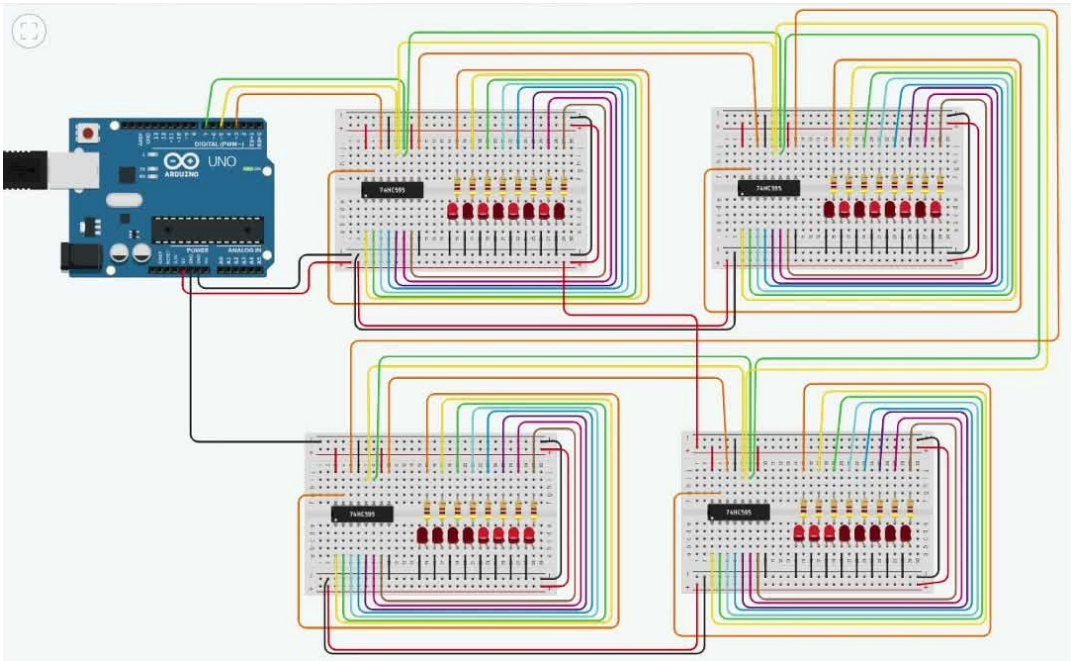


Figura 8: Quattro 74HC595 in cascata su Tinkercad: 32 LED pilotati con i soli tre fili di Arduino.

```
void loop()
{
  invia595(224);
  invia595(15);
  invia595(0x55);
  invia595(0xAA);
  // attivo RCLK per trasferire i dati dal registro a uscita
  carica(RCLK);
  delay(500);
}
```

`0x55` e `0xAA` sono due numeri scritti in **esadecimale** (prefisso **`0x`**): `0x55` = 85 = `B01010101` (un LED sì e uno no), `0xAA` = 170 = `B10101010` (l'alternanza opposta). Il primo numero inviato (224) finisce nel 595 **più lontano**, l'ultimo (0xAA) nel **primo**. Il resto del programma (`invia595` e `carica`) è **identico** a quello per due 595.

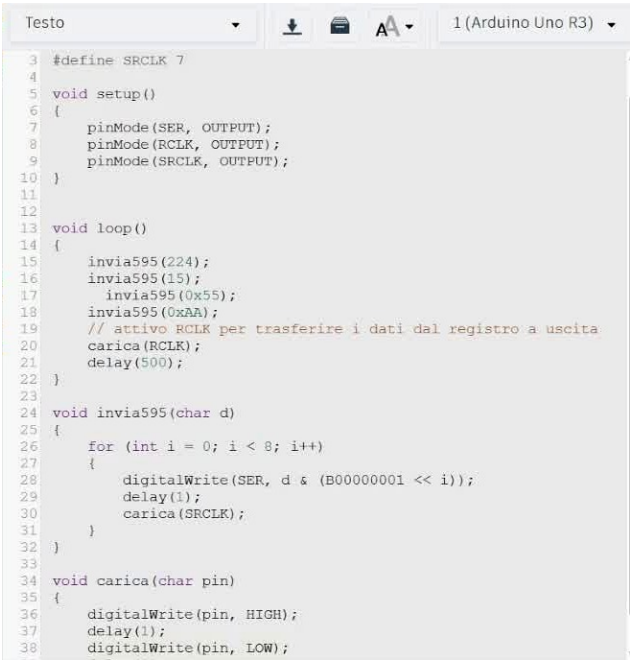


Figura 9: Il codice della soluzione nell'editor di Tinkercad: quattro `invia595` seguiti da un solo `carica(RCLK)`.

8 Cosa ricordare di questa lezione

- Una funzione ha un **tipo di ritorno**, un **nome** e dei **parametri**: `float ipotenusa(int a, int b)`. **`void`** vuol dire che **non restituisce niente**.
- Con **`return`** la funzione restituisce il risultato, che va messo in una variabile **del tipo giusto**: un `float` in un `int` perde la parte decimale.
- **`sqrt`** calcola la radice quadrata. $\sqrt{3^2+4^2}=5$, $\sqrt{9^2+12^2}=15$, $\sqrt{4^2+7^2}\approx 8{,}06$.
- Le funzioni fanno **scrivere meno** e quindi **sbagliare meno**; cambiando i dati in ingresso cambia il risultato.
- Per il computer **solo lo zero è falso**: `d & (B00000001 << i)` è "vero" se il bit `i` di `d` è 1. 181 = 10110101.

- **`invia595(d)`** spedisce gli 8 bit di `d`; **`carica(pin)`** dà un impulso sul piedino che le passiamo (SRCLK a ogni bit, RCLK per pubblicare).
- Un **array** raccoglie più numeri in un unico oggetto: `sequenza[i]` è l'elemento in posizione `i` (da 0). Con un `for` si scorre tutta la sequenza.
- **595 in cascata**: piedino **9** del primo al **14** del successivo, **11 e 12 in parallelo**, alimentazione a tutti. Il **primo** byte inviato finisce nel 595 **più lontano**.
- L'impulso su **RCLK** va dato **una volta sola, dopo tutti gli invii**: altrimenti i LED fanno **flickering**.

Esempio

Per la verifica di giovedì 5 novembre: la verifica copre **tutto il contenuto delle dispense** A01, A01bis, A02 e il firmware dell'anno scorso (LED, pulsanti, monitor seriale, funzioni, 595).